

State Management, Part 1

Lecture 4: debugging, Stateless vs Stateful, introducing state management

Who's teaching this course

Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro

Microsoft Teams course channel

rusudinu.com

45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with



TODAY

What you should leave with



Debug on purpose

Launch and attach DevTools, read the inspector, set breakpoints and step through Dart



The three trees

Widget, Element and RenderObject, and what setState actually triggers



Stateless vs Stateful

When a widget really needs a State object, and when it genuinely does not



Sharing state

Lifting state up, InheritedWidget, and why value equality decides rebuilds

PART 1 · DEBUGGING

Seeing what your app is actually doing

DevTools, breakpoints, and useful logging

The debugging toolbox



Hot reload

Injects changed source into the running VM and rebuilds. State objects survive.



Hot restart

Rebuilds from scratch and drops all state. Use it whenever initState or a global changed.



debugPrint & assert

Cheap, always available, and compiled out of release builds entirely.



Breakpoints

Stop on a line, inspect every variable in scope, step through the frame.



DevTools

Inspector, performance, CPU, memory, network: a browser tab attached to your app.



Read the error

Flutter's exception boxes name the widget and the constraint that failed.

print, debugPrint, and assert

```
print('value: $x');          // logcat truncates long lines
debugPrint('value: $x');    // throttled, nothing is dropped
assert(count >= 0, 'count negative: $count'); // debug only
if (kDebugMode) debugPrint(jsonEncode(payload));

// Three switches worth memorizing:
debugDumpApp();              // widget tree, as text
debugDumpRenderTree();      // sizes + constraints
debugPrintRebuildDirtyWidgets = true; // logs every rebuild
```

Why not just print?

Android's logcat silently drops lines beyond a length limit. `debugPrint` throttles output so nothing disappears.

`assert` takes a message. Use it for invariants you want to hear about in the lab and never in production.

`debugPrintRebuildDirtyWidgets` is the print that matters today: it names every widget that rebuilt, in build order.

Logging is a primary debugging tool. It is the only tool that works identically on a simulator, a real phone and a colleague's machine.

Launching and attaching DevTools

```
# Run in debug mode; the console prints the DevTools URL.  
flutter run  
  
# Flutter DevTools debugger and profiler is available at:  
# http://127.0.0.1:9100?uri=http://127.0.0.1:53412/AbCdEfGh=  
  
# Already running? Launch DevTools and paste that VM Service URI:  
dart devtools  
  
# From the IDE, skip all of it:  
# VS Code          Cmd/Ctrl+Shift+P -> "Dart: Open DevTools"  
# Android Studio   the "Flutter Inspector" tool window
```

Attach the tools before you need them. Performance work needs `flutter run --profile: debug` builds are JIT and assertion-heavy, so their timings mean nothing.

What each DevTools tab is for



Inspector

The live widget tree, the selected widget's properties, and the Layout Explorer for Row/Column overflow.



Performance

Frame timeline, UI vs raster thread, and exactly which frames blew the budget.



CPU Profiler

Where Dart time actually goes: a flame chart of your own functions.



Memory

Heap snapshots and allocation tracking. This is how you find the controller you forgot to dispose.



Network

HTTP and WebSocket traffic, request by request. We come back to this in Lecture 6.



Logging

The console stream, plus structured events and timeline entries.

The Widget Inspector

Select Widget Mode: tap a pixel on the running device and the inspector jumps to that widget, and your IDE jumps to the source line that built it.

The Layout Explorer shows flex factors and incoming constraints. It is where you find the cause of a RenderFlex overflow, the yellow-and-black stripes.

Track widget rebuilds counts how many times each widget rebuilt. That count is the measurement this lecture is about.

Highlight repaints wraps every layer in a rotating color. Anything flashing constantly is repainting when it should not be.

Try this in the lab

- 1 Run your todo app
- 2 Open the Inspector
- 3 Turn on Track widget rebuilds
- 4 Type one character into a text field
- 5 Count what rebuilt

You cannot reason about rebuilds from the source alone. The inspector shows you the ones that actually happen.

Frames, jank, and the performance overlay

16.7

ms at 60 Hz

one frame budget

8.3

ms at 120 Hz

most 2026 phones

2

threads to watch

UI (Dart) and raster (Impeller)

The performance overlay draws two bar charts: the UI thread, which runs your Dart build and layout, and the raster thread, where Impeller turns the layer tree into GPU commands. A bar that crosses the line is a dropped frame.

There is no fixed 16 ms rule. One frame budget is $1 \div \text{refresh rate}$, and most phones you will be graded on are 90–120 Hz.

The usual causes sit squarely in this lecture: `setState` called too high in the tree, expensive work inside `build()`, and missing `const`.

Breakpoints and stepping

Click the gutter in VS Code or Android Studio to set a breakpoint, then start the app with the debugger attached; plain flutter run in a terminal has no debugger.

Right-click a breakpoint to make it conditional: `todo.id == 42`. That is more precise than a print inside a loop.

`debugger()` from `dart:developer` breaks from code, which is useful inside a callback you cannot click on.

While paused you can hover any variable, edit it in the Variables pane, and evaluate arbitrary Dart in the console.

Step over

Run this line. Do not descend into the calls on it.

Step into

Descend into the call on this line.

Step out

Finish this function, stop back at the caller.

Resume

Run until the next breakpoint, or until the app exits.

Hot reload keeps your State objects; hot restart throws them away. If a change does not appear, that is nearly always the reason.

STATE

What changes, and who owns it

Three trees, setState, and why StatelessWidget does not “build once”

What counts as state

	Ephemeral: UI state	App: shared state
Lives in	one widget's State object	above every widget that reads it
Looks like	a checkbox tick, a page index, an animation, text being typed	the signed-in user, theme mode, the cart, a cached feed
Disposed when	the widget leaves the tree	never, ideally: it outlives navigation
Reach for	setState	lift it up, then a shared holder

Most state is ephemeral. If setState in one widget is enough, that is the correct answer, not a beginner's answer.

Widgets are immutable blueprints

A widget is a description of a piece of UI, not the UI itself. Once constructed, its fields never change.

Changing the screen means constructing a new description, never editing the old one.

That is what makes the framework's job cheap: comparing two immutable descriptions is fast, and whole subtrees can be skipped.

`const` goes further. The compiler canonicalizes a `const` widget, so the identical instance comes back every build, and the framework can prove nothing changed.

```
const Text('Target °C')
```

the same instance every build

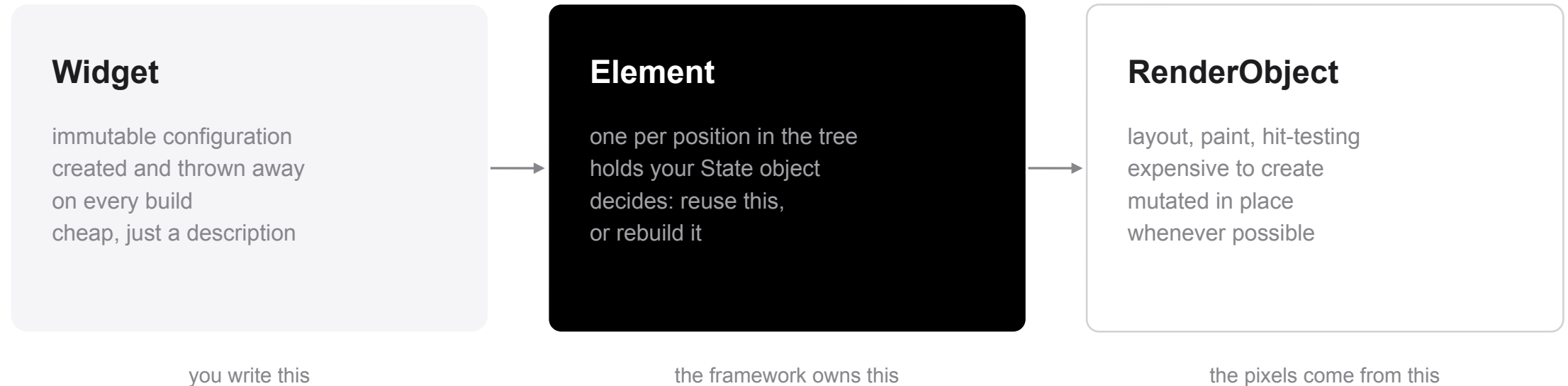
```
Text('Target °C')
```

a new instance every build

Immutability is what makes rebuilding cheap.

The three trees

Every Flutter app maintains three parallel trees. Almost every rebuild question is answered by knowing which one you are talking about.



You build the left tree. The middle tree decides how little of the right tree has to change.

What setState actually does

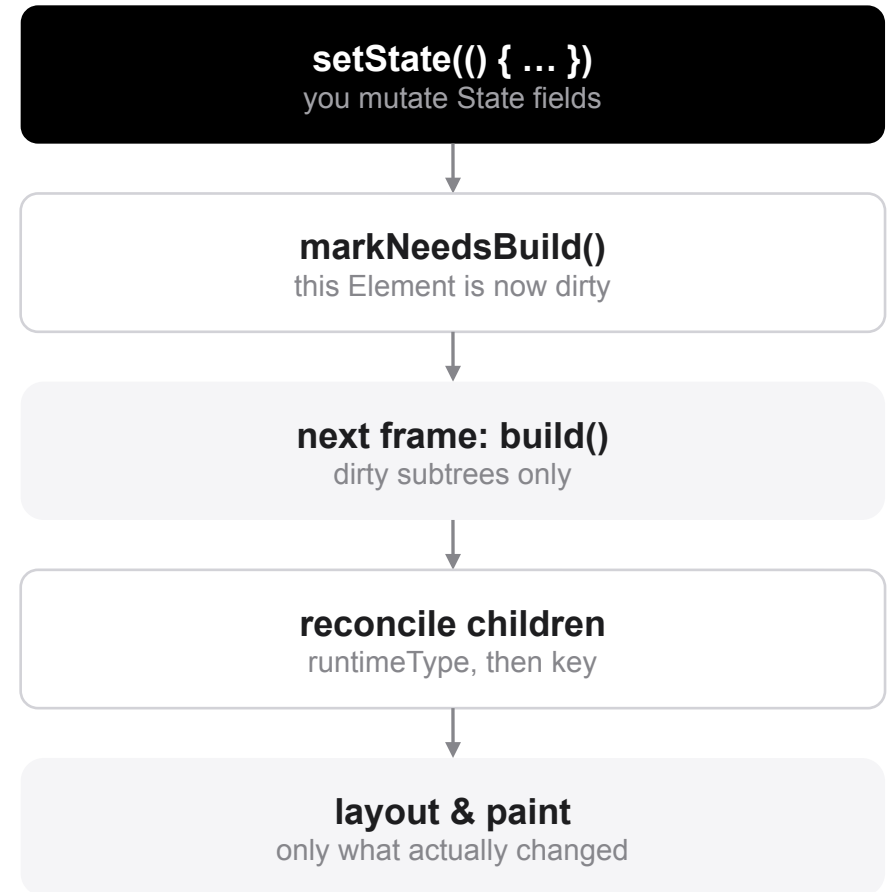
Flutter does not rebuild the widget tree every frame.

setState marks exactly one Element dirty and asks for a frame. On that frame, build() runs on the dirty subtrees, and on nothing else.

Sibling subtrees are never visited. Their Elements, State objects and RenderObjects are left completely untouched.

So the cost of a setState is the size of the subtree beneath it. Push it as far down the tree as you can.

Only dirty subtrees rebuild. That is the core of the performance model.



Reuse is decided by runtimeType, then key

When a parent rebuilds, each Element compares its old widget with the new one at the same position.

```
// Same runtimeType AND same key -> Element and State are reused.  
// Different runtimeType or key -> old subtree is disposed, new one built.  
  
ListView(  
  children: [  
    for (final todo in todos)  
      TodoTile(key: ValueKey(todo.id), todo: todo),  
  ],  
);  
  
// Without the key, Flutter matches children by POSITION.  
// Delete the first todo and the second row inherits the first row's  
// State: its scroll offset, its half-typed text, its animation.
```

Stateless vs Stateful, precisely

	StatelessWidget	StatefulWidget
Mutable state	none of its own	a separate, long-lived State object
build() runs	on insert, and on every parent rebuild	all of that, plus on every setState()
Lifecycle hooks	none	initState · didUpdateWidget · dispose
Genuinely fits	a label, an icon, a button that only calls a callback	a text field, an animation, a timer or stream subscription

A StatelessWidget does not “build once”. It rebuilds whenever its parent does; it simply holds no mutable state of its own. And a button is usually stateless: the state lives in whatever its callback changes.

Inside a State object

```
class _CounterFieldState extends State<CounterField> {  
  // Fields here survive rebuilds. The widget object does not.  
  final _controller = TextEditingController();  
  int _count = 0;  
  
  @override  
  void dispose() {  
    _controller.dispose();      // release what you created  
    super.dispose();  
  }  
  
  void _increment() => setState(() => _count++);  
  
  @override  
  Widget build(BuildContext context) => Text('$_count');  
}
```

Rules worth following

State lives in the State class, never on the widget. Widget fields are re-created on every rebuild.

Read the widget's fields through widget.something; the framework refreshes that reference for you.

Anything you create (controllers, timers, stream subscriptions) you dispose.

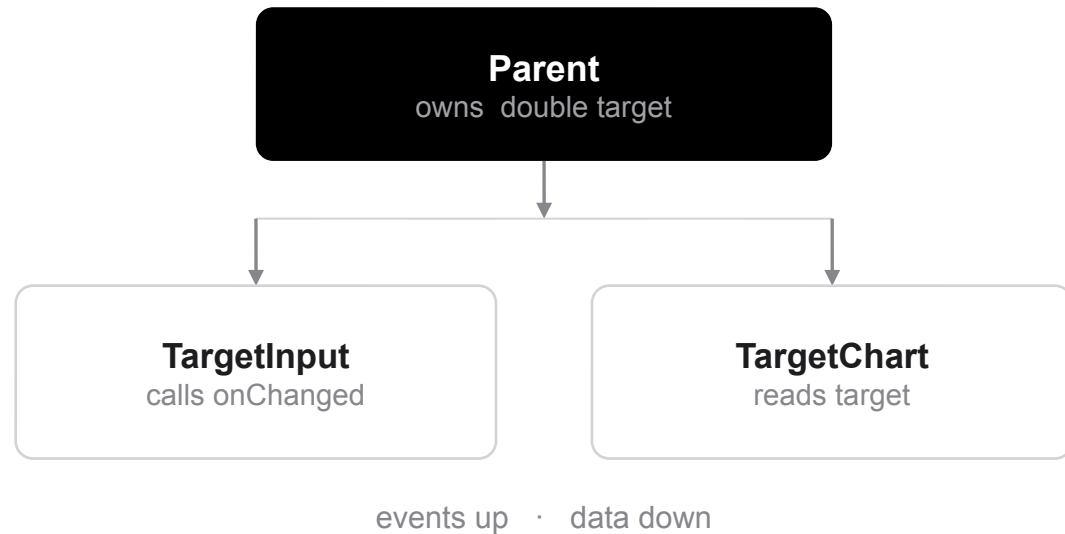
Never call setState from build(), and never after dispose(): check mounted after an await.

SHARING STATE

When one widget is not enough

Lifting state up · InheritedWidget · value equality

Lifting state up



Two widgets need the same value, so it moves to their nearest common ancestor. Data flows down as constructor arguments; events flow back up as callbacks.

The children stay reusable and know nothing about each other.

Lift too far and you get prop drilling: a value threaded through five widgets that do not care about it, just to reach the sixth.

That is precisely the problem the next slide solves.

Lifting state up is the default answer. Reach for a package only when the drilling becomes a real problem.

InheritedWidget: what Provider is built on

Put the value above the widgets that need it, and let them look it up directly instead of receiving it hand to hand.

```
class TargetScope extends InheritedWidget {  
  const TargetScope({super.key, required this.target, required super.child});  
  final double target;  
  
  // O(1) lookup, and it subscribes this context to future changes.  
  static double of(BuildContext context) =>  
    context.dependOnInheritedWidgetOfExactType<TargetScope>()!.target;  
  
  @override  
  bool updateShouldNotify(TargetScope old) => old.target != target;  
}  
  
// Anywhere below it in the tree, with no constructor plumbing:  
Text('${TargetScope.of(context)} °C');
```

Value equality: why Equatable exists

```
class CounterState extends Equatable {  
  const CounterState({required this.count, required this.status});  
  final int count;  
  final LoadStatus status;  
  
  // == and hashCode are generated from exactly this list.  
  @override  
  List<Object?> get props => [count, status];  
}  
  
const a = CounterState(count: 1, status: LoadStatus.ready);  
const b = CounterState(count: 1, status: LoadStatus.ready);  
  
a == b;    // true  with Equatable  
          // false without it: Dart's default == is identity
```

Why it matters

Every state consumer asks one question: is this state different from the last one?

Identity equality answers “yes” every time. Two structurally identical objects are still two objects, so the UI rebuilds when nothing changed.

The mirror failure: mutate a state object in place and re-emit the same instance, and identity answers “no”, so the UI does not rebuild even though the data did change.

Hence the rule: state classes are immutable and compared by value.

Add a field, add it to props. Forgetting is the classic bug.

Pick the smallest tool that works

	setState	Lifting up	InheritedWidget	Provider / Riverpod	BLoC / Cubit
Scope	one widget	a few siblings	one subtree	whole app	whole app
Mechanism	mark dirty	callbacks up	lookup + notify	wraps Inherited	events to states
Costs you	nothing	prop drilling	boilerplate	a dependency	a dependency + a pattern
In this course	today	today	today	named only	the project

Provider is a thin, ergonomic wrapper around InheritedWidget. Riverpod is its successor: the same idea without needing a BuildContext, and checked at compile time. BLoC turns state into a stream of states driven by a stream of events.

We name all three so you recognize them in a code review. Do not adopt a package you cannot explain.

This is Part 1. setState, lifting up and InheritedWidget carry most screens. The last column, BLoC and Cubit, is what the graded project asks for: the same ideas, expressed as events in and states out.

Before next week



Recap

DevTools first: look at what the app is doing before changing it

Three trees: you rebuild widgets, the framework reuses Elements

Only dirty subtrees rebuild; reuse is runtimeType, then key

setState for ephemeral state, lift it up when siblings share it

Value equality decides whether the UI rebuilds at all



This week

Run your lab app under DevTools and attach it from the printed URL

Turn on Track widget rebuilds; find one rebuild you did not expect

Add GlobalKey to a list of stateful tiles and watch the behavior change

Make one model class extend Equatable and write a == test for it



Read more

docs.flutter.dev/tools/devtools

docs.flutter.dev/data-and-backend/state-mgmt/intro

api.flutter.dev: InheritedWidget, State, Key

pub.dev/packages/equatable